



Universidade do Minho
Escola de Engenharia

Relatório Trabalho Prático Nº2 (PL32)

Tomé Francisco Amorim Moreira - a111907

Miguel Pedro Ribeiro Correia - a111773

José Pedro Fernandes da Costa - a109807

8 de maio de 2026

Índice Geral

1	Arquitetura do sniffer	2
1.1	Visão Geral	2
1.2	Fluxo de Execução	2
1.3	Módulo de Captura (sniffer.py)	3
1.4	Parser de Pacotes (parser.py)	3
1.5	Modelo de Dados (models.py)	3
1.6	Estatísticas (stats.py)	3
1.7	Handlers por Protocolo	4
1.8	Formatação (formatter.py)	4
1.9	Modo de Análise (analysis.py)	4
1.10	Filtragem (filters.py)	5
1.11	Conclusão	6
2	Catálogo e Análise de Protocolos	6
2.1	ICMP — Internet Control Message Protocol	6
2.2	TCP — Transmission Control Protocol	7
2.3	ARP — Address Resolution Protocol	8
2.4	IPv4 — Internet Protocol version 4	9
2.5	IEEE 802.11 / WiFi — Wireless LAN Frames	10
2.6	DHCP — Dynamic Host Configuration Protocol	12
3	Lista de filtros e parâmetros de entrada disponíveis	13
3.1	Lista de filtros	13
3.2	Parâmetros de entrada e modo de utilização	15
4	Topologia de Rede adotada no CORE	16
5	Testes para a validação do funcionamento do sniffer	16
5.1	Validação da correta análise do ICMP	16
5.2	Validação da correta análise do TCP	17
5.3	Validação da correta análise do ARP	18
5.4	Validação da correta análise do IPv4	19
5.5	Validação da correta análise IEEE 802.11 (WiFi)	21
5.6	Validação da correta análise do DHCP	22
6	Análise em Interface de Rede Real	23
6.1	Observações	23
6.2	Resultados	24
7	Obstáculos Encontrados	24
8	Limitações da Aplicação Desenvolvida	25
9	Conclusão do Trabalho Efetuado	25

1 Arquitetura do sniffer

O sniffer desenvolvido segue uma arquitetura modular, onde cada componente tem uma responsabilidade bem definida. Esta abordagem facilita a manutenção, extensão e organização do código.

1.1 Visão Geral

O sistema é composto por vários módulos principais:

- **main.py** – ponto de entrada da aplicação
- **sniffer.py** – captura e processamento de pacotes
- **parser.py** – extração de informação dos pacotes
- **stats.py** – cálculo de estatísticas globais
- **formatter.py** – formatação da saída
- **analysis.py** – modo interativo de análise
- **handlers/** – análise detalhada por protocolo
- **filters.py** – filtragem de pacotes

1.2 Fluxo de Execução

A execução inicia-se no módulo **main.py**, onde são processados os argumentos da linha de comandos (interface, modo de execução, ficheiro de saída e nível de verbosidade).

De seguida, é invocada a função **start_sniffing()**, responsável por iniciar a captura de pacotes através da biblioteca Scapy.

Cada pacote capturado segue o seguinte fluxo:

1. Captura do pacote (**sniffer.py**)
2. Parsing e extração de informação (**parser.py**)
3. Atualização de estatísticas globais (**stats.py**)
4. Encaminhamento para handlers específicos por protocolo
5. Formatação da saída (**formatter.py**)
6. Impressão ou escrita em ficheiro

1.3 Módulo de Captura (`sniffer.py`)

O módulo `sniffer.py` é responsável por:

- Capturar pacotes em tempo real usando `sniff()`
- Manter uma lista global de pacotes capturados
- Invocar o parser para extrair informação relevante
- Encaminhar pacotes para handlers específicos (ICMP, TCP, ARP, DHCP, WiFi, etc.)

Este módulo funciona como o núcleo do sistema, coordenando os restantes componentes.

1.4 Parser de Pacotes (`parser.py`)

O `parser.py` extrai informação estruturada de cada pacote e converte-a numa instância da classe `PacketInfo`.

As principais responsabilidades incluem:

- Identificação das camadas (rede, transporte, aplicação)
- Determinação do protocolo principal
- Extração de endereços MAC e IP
- Definição de um resumo textual do pacote

1.5 Modelo de Dados (`models.py`)

A estrutura `PacketInfo` representa cada pacote capturado e contém:

- Timestamp
- Interface
- Protocolo principal
- Endereços MAC e IP
- Tamanho do pacote
- Resumo descritivo

Esta abstração permite desacoplar o parsing da análise.

1.6 Estatísticas (`stats.py`)

O módulo `stats.py` mantém estatísticas globais da captura, incluindo:

- Número total de pacotes
- Volume total de tráfego
- Distribuição por protocolo
- Taxa média de transferência

As estatísticas são atualizadas em tempo real e apresentadas no final da captura.

1.7 Handlers por Protocolo

Os handlers, organizados na pasta **handlers/**, são responsáveis pela análise detalhada de cada protocolo identificado durante a captura. Cada handler implementa lógica específica e mantém estruturas de dados próprias para suportar análise posterior.

Ao contrário de uma simples inspeção de pacotes, estes módulos permitem reconstruir interações e extrair métricas relevantes da comunicação de rede.

Exemplos de Análise Implementada

- **ICMP** Emparelhamento de mensagens *Echo Request* e *Echo Reply*, permitindo calcular o RTT (Round-Trip Time) entre dois hosts. São mantidas tabelas de pedidos pendentes e listas de pares completos com estatísticas associadas (mínimo, máximo e média).
- **ARP** Reconstrução de pares *request/reply*, identificação de pacotes *gratuitous ARP* e cálculo do tempo de resposta entre pedidos e respostas. Permite observar resolução de endereços IP para MAC dentro da rede local.
- **TCP** Análise do processo de estabelecimento de ligações TCP através do *three-way handshake* (SYN, SYN-ACK, ACK). São identificadas conexões completas, calculado o RTT inicial e registados eventos de término (FIN) ou interrupção (RST).
- **DHCP** Reconstrução de transações completas com base no identificador XID. Permite acompanhar as fases do protocolo (Discover, Offer, Request, ACK) e medir o tempo total de atribuição de endereço IP.
- **IPv4** Detecção e análise de fragmentação IP, agrupando fragmentos por identificador. São apresentados offsets, tamanhos, tempos relativos de chegada e verificado se o datagrama foi completamente recebido.
- **WiFi (IEEE 802.11)** Análise de frames de gestão, controlo e dados. Permite identificar redes ativas (beacons), dispositivos mais ativos e padrões de utilização (ex: scanning vs tráfego normal).

Cada handler mantém estruturas internas (tabelas, listas e estatísticas) que permitem uma análise posterior mais aprofundada no modo interativo.

1.8 Formatação (formatter.py)

Este módulo é responsável por apresentar os dados ao utilizador em dois formatos:

- **Verbose** – informação detalhada
- **Compact** – formato resumido

1.9 Modo de Análise (analysis.py)

Após a fase de captura, o sistema entra num modo interativo que permite ao utilizador explorar os dados recolhidos.

Este modo funciona como uma interface de linha de comandos onde é possível executar diferentes tipos de análise sobre os pacotes armazenados.

Comandos Disponíveis

- **icmp** Apresenta pares de mensagens ICMP (Echo Request/Reply) com cálculo de RTT.
- **arp** Mostra pares de pedidos e respostas ARP, permitindo observar a resolução de endereços na rede.
- **tcp** Lista conexões TCP estabelecidas, incluindo endereços, portas e RTT do handshake.
- **dhcp** Apresenta fluxos de transações DHCP organizados por XID, incluindo tempos relativos entre mensagens.
- **ipv4** Exibe informação detalhada sobre pacotes IPv4 (flags, TTL, tamanho, protocolo).
- **fragments ipv4** Mostra grupos de fragmentos IP, permitindo analisar fragmentação e reconstrução de datagramas.
- **wifi** Apresenta análise de tráfego WiFi, incluindo dispositivos ativos e redes detectadas.

Comandos de Estatísticas

- **status icmp** – estatísticas de RTT e contagem de pedidos/respostas
- **status arp** – distribuição de pedidos, respostas e ARP gratuitos
- **status tcp** – contagem de eventos TCP (SYN, ACK, FIN, RST) e conexões estabelecidas
- **status wifi** – distribuição de tipos de frames e atividade na rede sem fios

Filtragem de Pacotes

O modo de análise permite ainda aplicar filtros sobre os pacotes capturados:

- **filter ip <endereço>** – filtra pacotes por endereço IP
- **filter mac <endereço>** – filtra por endereço MAC
- **filter proto <protocolo>** – filtra por protocolo (ex: TCP, ICMP, ARP)

1.10 Filtragem (filters.py)

Permite filtrar pacotes com base em:

- Endereço IP
- Endereço MAC
- Protocolo

1.11 Conclusão

A arquitetura modular adotada permite separar claramente as responsabilidades entre captura, processamento, análise e apresentação. Esta abordagem facilita a extensibilidade do sistema, permitindo adicionar novos protocolos ou funcionalidades com impacto reduzido no restante código.

2 Catalogação e Análise de Protocolos

O *packet sniffer* desenvolvido utiliza a biblioteca Scapy para capturar e dissecar pacotes de rede. A identificação dos protocolos é feita através das camadas presentes em cada pacote, sendo o handler invocado conforme o tipo detetado.

2.1 ICMP — Internet Control Message Protocol

2.1.1 Identificação e tipos catalogados

O protocolo ICMP é identificado pela presença da camada ICMP no pacote. O handler analisa o campo `icmp.type` para distinguir dois tipos de mensagem:

- **Echo Request** (`type = 8`) — pedido de ping enviado pelo host;
- **Echo Reply** (`type = 0`) — resposta ao pedido, enviada pelo destino.

2.1.2 Estruturas de dados

A análise recorre a três estruturas principais:

- `icmp_table` — dicionário que armazena os requests pendentes, indexado pela tupla (`src_ip`, `dst_ip`, `icmp.id`, `icmp.seq`);
- `icmp_pairs` — lista de pares completos (request + reply) com o RTT calculado;
- `icmp_stats` — contadores globais de requests, replies e lista de RTTs.

2.1.3 Análise e cálculo do RTT

Quando um Echo Request é recebido, é registado em `icmp_table` com a sua chave identificadora. Ao chegar um Echo Reply, a chave é invertida (sentido oposto do tráfego) e procurada nos pedidos pendentes:

```
1  if icmp.type == 8:                                     # Echo Request
2      icmp_table[key] = info
3
4  elif icmp.type == 0:                                   # Echo Reply
5      reverse_key = (dst_ip, src_ip, id, seq)
6      if reverse_key in icmp_table:
7          rtt_ms = (info.timestamp - request.timestamp) * 1000
8          icmp_pairs.append({ ..., "rtt": rtt_ms })
```

O RTT é obtido pela diferença entre os timestamps de captura do reply e do request correspondente, convertida para milissegundos.

2.2 TCP — Transmission Control Protocol

2.2.1 Identificação e flags catalogadas

O TCP é identificado pela camada TCP do pacote. A análise baseia-se no campo `tcp.flags`, que reflete o estado de cada segmento:

Flag	Significado	Ação
"S"	SYN	Regista novo flow em <code>tcp_flows</code>
"SA"	SYN-ACK	Associa ao flow com chave invertida
"A"	ACK	Finaliza handshake; calcula RTT
"F" in flags	FIN	Encerramento normal; remove flow
"R" in flags	RST	Encerramento forçado; remove flow

Tabela 1: Flags TCP catalogadas pelo sniffer

2.2.2 Estruturas de dados

A análise TCP recorre a três estruturas principais:

- `tcp_flows` — dicionário que armazena ligações em progresso (handshake incompleto), indexado pela tupla (`src_ip`, `sport`, `dst_ip`, `dport`);
- `tcp_connections` — lista de ligações completamente estabelecidas, com RTT calculado;
- `tcp_stats` — contadores globais por flag (SYN, SYN-ACK, ACK, FIN, RST) e lista de RTTs.

2.2.3 Reconstrução do three-way handshake

O sniffer reconstrói o handshake TCP estado a estado, usando `tcp_flows` como estado intermédio indexado por (`src_ip`, `sport`, `dst_ip`, `dport`):

```
1  if flags == "S":                                     # SYN
2      tcp_flows[key] = {"syn": info}
3
4  elif flags == "SA":                                   # SYN-ACK
5      if reverse_key in tcp_flows:
6          tcp_flows[reverse_key]["syn_ack"] = info
7
8  elif flags == "A" and "syn_ack" in tcp_flows[key]:   # ACK
9      rtt_ms = (syn_ack.timestamp - syn.timestamp) * 1000
10     tcp_connections.append({ ..., "rtt": rtt_ms })
```

O RTT medido corresponde ao intervalo entre o SYN e o SYN-ACK, representando a latência de resposta do servidor. Os segmentos FIN e RST apenas removem o flow dos pendentes, sem contribuir para métricas.

2.3 ARP — Address Resolution Protocol

2.3.1 Identificação e tipos catalogados

O protocolo ARP é identificado pela presença da camada ARP no pacote. A distinção entre tipos de mensagem é feita através do campo `arp.op`:

- **Request** (`op = 1`) — pedido de resolução de endereço (“quem tem este IP?”);
- **Reply** (`op = 2`) — resposta contendo o endereço MAC associado;
- **Gratuitous** (`psrc == pdst`) — anúncio espontâneo de mapeamento IP–MAC.

Os pacotes ARP Gratuitous não são considerados no emparelhamento request/reply, sendo apenas contabilizados em estatísticas globais.

2.3.2 Semântica dos campos `psrc` e `pdst`

Ao contrário de ICMP e TCP, o ARP opera ao nível da *link layer*, utilizando uma semântica própria para endereços IP:

- `psrc` — IP do emissor do pacote;
- `pdst` — IP alvo cuja resolução é pretendida.

Num ARP Request, `psrc` corresponde ao cliente e `pdst` ao destino procurado. Na resposta (ARP Reply), estes valores surgem invertidos, permitindo a correlação direta entre mensagens.

2.3.3 Estruturas de dados

A análise ARP recorre a três estruturas principais:

- `arp_table` — dicionário que armazena requests pendentes, indexado pela tupla (`psrc`, `pdst`);
- `arp_pairs` — lista de pares completos (request + reply) com RTT associado;
- `arp_stats` — contadores globais (requests, replies, gratuitous) e lista de RTTs.

2.3.4 Emparelhamento e cálculo do RTT

O handler ARP segue uma lógica semelhante à do ICMP, baseada no armazenamento temporário de requests e posterior correspondência com replies.

```
1 if arp.op == 1:                                # ARP Request
2     arp_stats["requests"] += 1
3     arp_table[key] = info                        # key = (psrc, pdst)
4
5 elif arp.op == 2:                                # ARP Reply
6     arp_stats["replies"] += 1
7     reverse_key = (arp.pdst, arp.psrc)
8
```

```

9     if reverse_key in arp_table:
10         request = arp_table.pop(reverse_key)
11         rtt_ms = (info.timestamp - request.timestamp) * 1000
12         arp_stats["rtts"].append(rtt_ms)
13         arp_pairs.append({ ..., "rtt": rtt_ms })

```

Tal como nos restantes protocolos analisados, o RTT é obtido pela diferença entre os instantes de captura do request e do reply correspondente, expresso em milissegundos. Dado que o ARP envolve apenas duas mensagens, o processo de correlação é mais simples do que no TCP, não existindo estados intermédios nem controlo de ligação.

2.4 IPv4 — Internet Protocol version 4

2.4.1 Identificação e tipos catalogados

O protocolo IPv4 é identificado pela presença da camada IP no pacote. O handler analisa o cabeçalho IPv4 na sua totalidade, extraindo os campos relevantes para análise, com especial atenção à fragmentação de datagramas.

Um pacote IPv4 é considerado fragmentado quando se verifica uma das seguintes condições:

- **More Fragments** (MF = 1) — indica que existem mais fragmentos a seguir;
- **Fragment Offset** > 0 — indica que o pacote é um fragmento intermédio ou final.

2.4.2 Estruturas de dados

A análise IPv4 recorre a duas estruturas principais:

- **ipv4_fragments** — dicionário que agrupa fragmentos pelo campo ID do cabeçalho IPv4, permitindo associar todos os fragmentos pertencentes ao mesmo datagrama original;
- **ipv4_packetsInfo** — lista com todos os pacotes IPv4 capturados, incluindo fragmentados e não fragmentados, para análise geral do cabeçalho.

2.4.3 Análise do cabeçalho e deteção de fragmentação

Para cada pacote IPv4 recebido, o handler extrai os campos do cabeçalho e verifica se se trata de um fragmento:

```

1  mf = int(ipv4.flags) & 0x1
2  is_fragment = (ipv4.frag > 0) or (mf == 1)
3
4  if is_fragment:
5      if key not in ipv4_fragments:
6          ipv4_fragments[key] = []
7          ipv4_fragments[key].append((ipv4, info))
8
9  ipv4_packetsInfo.append((ipv4, info))

```

Os fragmentos são agrupados pelo campo ID do cabeçalho, que é comum a todos os fragmentos do mesmo datagrama original. A ordenação por *Fragment Offset* permite reconstituir a sequência correta dos fragmentos e determinar se o datagrama está completo — o último fragmento é identificado por `MF = 0`.

Os campos extraídos para cada pacote incluem o endereço de origem e destino, o protocolo encapsulado, o TTL, o tamanho total, o *checksum* e as flags `DF`, `MF` e `Res`.

2.5 IEEE 802.11 / WiFi — Wireless LAN Frames

2.5.1 Identificação e categorias catalogadas

O tráfego WiFi é identificado pela presença da camada `Dot11` no pacote. O handler analisa o campo `dot11.type`, que permite classificar cada frame em três categorias principais:

- **Management** (`type = 0`) — frames de controlo lógico da rede, como Beacon, Probe, Association e Authentication;
- **Control** (`type = 1`) — frames auxiliares de acesso ao meio, acknowledgements e controlo MAC;
- **Data** (`type = 2`) — frames que transportam dados úteis entre clientes e ponto de acesso.

Para frames de gestão, o campo `dot11.subtype` é ainda utilizado para distinguir operações específicas:

Subtype	Nome	Função
8	Beacon	Anúncio periódico da rede WiFi
4	Probe Request	Procura ativa de redes disponíveis
5	Probe Response	Resposta a Probe Request
11	Authentication	Início de autenticação
0	Association Request	Pedido de associação ao AP
1	Association Response	Resposta ao pedido de associação

Tabela 2: Subtipos IEEE 802.11 catalogados pelo sniffer

2.5.2 Estruturas de dados

A análise WiFi utiliza quatro estruturas principais:

- `wifi_frames` — lista cronológica com informação resumida de cada frame capturado;
- `wifi_stats` — contadores globais por categoria (`management`, `control`, `data`);
- `device_activity` — dicionário com número de frames emitidos por cada endereço MAC origem;
- `bssid Beacons` — dicionário com contagem de beacons observados por BSSID.

2.5.3 Classificação do tráfego

Cada frame recebido é analisado segundo o valor de `dot11.type`, atualizando os contadores globais:

```
1 if dot11.type == 0:
2     wifi_stats["management"] += 1
3 elif dot11.type == 1:
4     wifi_stats["control"] += 1
5 elif dot11.type == 2:
6     wifi_stats["data"] += 1
```

Esta distribuição permite caracterizar o estado da rede. Um volume elevado de frames de dados indica utilização ativa, enquanto predominância de management frames sugere scanning, associação ou baixa atividade.

2.5.4 Detecção de dispositivos ativos

O endereço MAC origem (`addr2`) é utilizado para contabilizar atividade por dispositivo:

```
1 src = dot11.addr2
2 if src:
3     device_activity[src] =
4         device_activity.get(src, 0) + 1
```

Desta forma é possível identificar os equipamentos mais ativos durante a captura.

2.5.5 Identificação de redes visíveis

Quando o frame corresponde a um Beacon (`type = 0` e `subtype = 8`), o BSSID da rede é registado:

```
1 if dot11.type == 0 and dot11.subtype == 8:
2     bssid = dot11.addr3
3     if bssid:
4         bssid Beacons[bssid] =
5             bssid_beacons.get(bssid, 0) + 1
```

Isto permite enumerar redes WiFi detetadas e medir a frequência com que anunciam a sua presença.

2.5.6 Extração de metadados

Para cada frame é ainda guardado um resumo contendo timestamp, tipo, subtipo, endereços MAC e SSID (quando presente no elemento `Dot11Elt`):

```
1 frame_info = {
2     "timestamp": info.timestamp,
3     "type": dot11.type,
4     "subtype": dot11.subtype,
5     "src": dot11.addr2,
```

```

6     "dst": dot11.addr1,
7     "bssid": dot11.addr3,
8     "ssid": None
9 }

```

Caso o SSID esteja oculto ou não seja decodificável, o sistema apresenta o valor `<hidden>`.

2.5.7 Avaliação da qualidade da ligação

O programa calcula ainda a proporção de frames de dados relativamente ao total capturado:

```

1 data_ratio = wifi_stats["data"] / total

```

Com base nesse valor, a ligação é classificada como:

- **High** — utilização normal, predominância de tráfego útil;
- **Medium** — mistura entre gestão e dados;
- **Low** — rede inativa ou em scanning.

Esta métrica fornece uma estimativa simples do nível de utilização observado na interface sem fios.

2.6 DHCP — Dynamic Host Configuration Protocol

2.6.1 Identificação e tipos catalogados

O protocolo DHCP é identificado pela presença da camada DHCP no pacote. O handler analisa o campo `message-type` nas opções DHCP para distinguir os diferentes tipos de mensagem que compõem a negociação:

- **Discover** — enviado pelo host em broadcast para localizar servidores DHCP disponíveis;
- **Offer** — resposta do servidor com uma proposta de endereço IP;
- **Request** — confirmação do host de que aceita a oferta;
- **ACK** — confirmação final do servidor, concluindo a atribuição do endereço.

Esta sequência é conhecida como **DORA** (*Discover, Offer, Request, ACK*).

2.6.2 Estruturas de dados

A análise DHCP recorre a duas estruturas principais:

- `dhcp_transactions` — dicionário que agrupa todas as mensagens de uma transação pelo campo `XID` (*Transaction ID*), gerado aleatoriamente pelo host e partilhado por todas as mensagens da mesma negociação;
- `dhcp_infos` — lista de transações processadas, com os tempos relativos de cada fase calculados em relação ao *Discover*.

2.6.3 Agrupamento e análise das transações

Ao contrário do ICMP e ARP, o DHCP não envolve um simples par request/reply, mas sim uma sequência de até quatro mensagens. Por isso, o agrupamento é feito pelo XID e o processamento é realizado numa fase posterior à captura:

```
1 def handle_dhcp(packet, info):
2     dhcp = packet[B00TP]
3     key = dhcp.xid
4     if key not in dhcp_transactions:
5         dhcp_transactions[key] = []
6     dhcp_transactions[key].append(info)
```

Após a captura, a função `process_transactions()` percorre cada transação, ordena as mensagens por timestamp e calcula os tempos relativos de cada fase em relação ao *Discover*, que serve como instante de referência (`base_time = 0`):

```
1 if p.summary == "DHCP discover":
2     base_time = t
3     info["discover"] = 0
4
5 elif p.summary == "DHCP offer" and base_time is not None:
6     info["offer"] = (t - base_time) * 1000
7
8 elif p.summary == "DHCP ack" and base_time:
9     info["ack"] = (t - base_time) * 1000
10    info["total_time"] = (t - base_time) * 1000
```

O tempo total da negociação corresponde ao intervalo entre o *Discover* e o *ACK*, expresso em milissegundos. Transações incompletas — nas quais nem todas as fases foram observadas — são igualmente registadas, com os campos em falta marcados como `None`.

3 Lista de filtros e parâmetros de entrada disponíveis

3.1 Lista de filtros

Após a terminação da captura de pacotes, o *packet sniffer* disponibiliza um conjunto de filtros que permitem ao utilizador analisar de forma seletiva o tráfego previamente registado. Estes filtros são aplicados sobre a lista de estruturas `PacketInfo`, construída durante a fase de captura.

3.1.1 Filtro por protocolo

O filtro por protocolo permite seleccionar apenas os pacotes cujo protocolo principal corresponde ao valor indicado. A comparação é feita com base no atributo `top_protocol`, previamente determinado durante o parsing do pacote.

Adicionalmente, para os casos de IPv4 e IPv6, a distinção é feita com base no formato do endereço IP:

- Endereços contendo “.” são considerados IPv4;

- Endereços contendo “:” são considerados IPv6.

```
1 def filter_by_protocol(packets, proto):
2     proto = proto.upper()
3
4     return [
5         p for p in packets
6         if (
7             p.top_protocol.upper() == proto
8             or (proto == "IPv4" and hasattr(p, "src_ip") and p.
9                 src_ip and "." in p.src_ip)
10            or (proto == "IPv6" and hasattr(p, "src_ip") and p.
11                src_ip and ":" in p.src_ip)
12        )
13    ]
```

3.1.2 Filtro por endereço IP

O filtro por IP seleciona todos os pacotes em que o endereço fornecido corresponde ao endereço de origem ou de destino. Este filtro é útil para analisar toda a comunicação associada a um determinado nó da rede.

```
1 def filter_by_ip(packets, ip):
2     return [
3         p for p in packets
4         if p.src_ip == ip or p.dst_ip == ip
5     ]
```

3.1.3 Filtro por endereço MAC

O filtro por MAC funciona de forma análoga ao filtro por IP, mas ao nível da camada de ligação de dados. Permite identificar todo o tráfego associado a um determinado dispositivo físico na rede.

```
1 def filter_by_mac(packets, mac):
2     return [
3         p for p in packets
4         if p.src_mac == mac or p.dst_mac == mac
5     ]
```

3.1.4 Utilização dos filtros

Os filtros são invocados em modo de análise através de comandos interativos, como por exemplo:

- `filter proto tcp`
- `filter ip 10.0.0.1`

- `filter mac 00:11:22:33:44:55`

Estes comandos permitem ao utilizador restringir dinamicamente o conjunto de pacotes apresentados, facilitando a inspeção de fluxos específicos e a identificação de padrões de comunicação.

3.2 Parâmetros de entrada e modo de utilização

O *packet sniffer* desenvolvido permite a configuração do seu comportamento através de parâmetros de entrada fornecidos no momento da execução. Estes parâmetros são processados com recurso ao módulo `argparse`, possibilitando uma interface de linha de comandos simples e intuitiva.

Os parâmetros disponíveis são os seguintes:

- **Interface de rede** (`-i` ou `-interface`) — parâmetro obrigatório que especifica a interface de rede onde será realizada a captura de pacotes (ex.: `eth0`);
- **Modo de execução** (`-m` ou `-mode`) — define a forma como os dados capturados são apresentados ou armazenados. Este parâmetro admite três valores:
 - `live` — os pacotes são apresentados em tempo real na consola;
 - `log` — os pacotes são guardados num ficheiro, sem saída para a consola;
 - `both` — combina os dois modos anteriores, apresentando os pacotes em tempo real e guardando-os simultaneamente;

Por omissão, o modo selecionado é `live`;

- **Ficheiro de saída** (`-o` ou `-output`) — especifica o caminho do ficheiro onde os pacotes serão guardados quando o modo de execução inclui `log`. Caso não seja fornecido, não será criado qualquer ficheiro de saída.
- **Verbosity** (`-v` ou `-verbosity`) — define o nível de detalhe da informação apresentada. Este parâmetro admite dois valores:
 - `verbose` — apresenta informação detalhada sobre os pacotes capturados;
 - `compact` — apresenta apenas informação essencial e resumida;

Por omissão, o modo selecionado é `verbose`.

A execução do programa segue o formato geral:

```
python3 main.py -i <interface> [-m <mode>] [-o <ficheiro>] [-v <verbosity>]
```

Por exemplo, o seguinte comando inicia a captura na interface `eth0`, apresentando os pacotes na consola e guardando-os simultaneamente num ficheiro, de forma verbosa:

```
python3 main.py -i eth0 -m both -o output.txt -v verbosity
```

Estes parâmetros permitem adaptar o funcionamento do sniffer a diferentes cenários, desde monitorização em tempo real até recolha de dados para análise posterior.

4 Topologia de Rede adotada no CORE

Foi construída uma topologia de rede no CORE para permitir a geração e captura de tráfego num ambiente controlado. A topologia é composta por três nós finais (Joana, Júlio e João) interligados através de um router central (UMinhoR1), todos pertencentes à mesma sub-rede IPv4.

Cada host está configurado com um endereço IP na rede 10.0.2.0/24, permitindo comunicação direta entre si através do router. Esta estrutura simples, em forma de estrela, facilita a observação do tráfego de rede, uma vez que todo o encaminhamento passa pelo nó central.

A topologia adotada permite a identificação de vários protocolos da pilha TCP/IP. O protocolo ARP pode ser observado na resolução de endereços IP para MAC dentro da rede local. O DHCP pode ser identificado no processo de atribuição dinâmica de endereços IP (quando configurado). O ICMP é visível através de testes de conectividade, como o comando *ping*. O protocolo IPv4 está presente em todo o encapsulamento dos pacotes. O TCP pode ser analisado em comunicações orientadas à ligação, como transferências de dados entre hosts. Por fim, embora o meio físico seja simulado, o comportamento de interfaces de rede (como Wi-Fi no CORE) pode ser parcialmente observado ao nível de configuração e transmissão.

Apesar de nem todos os protocolos poderem ser explorados em profundidade nesta topologia, esta configuração é suficiente para demonstrar e analisar o funcionamento da maioria dos protocolos fundamentais de rede, sendo normal que alguns não sejam totalmente testáveis devido às limitações do ambiente simulado.

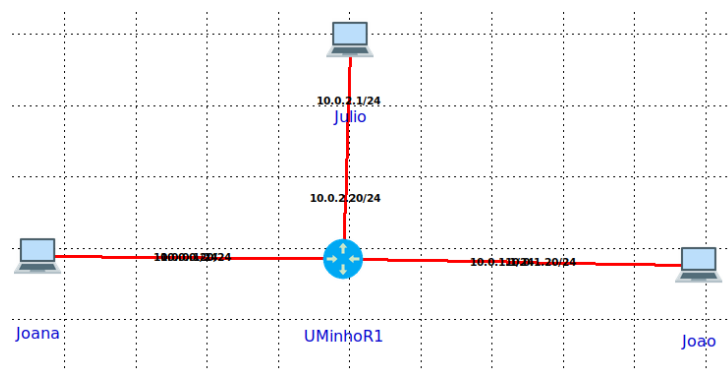


Figura 1: Topologia de rede implementada

5 Testes para a validação do funcionamento do sniffer

5.1 Validação da correta análise do ICMP

Para validar o correto funcionamento do sniffer na deteção e interpretação de pacotes ARP, foi realizado um teste no ambiente CORE, com base na topologia de rede apresentada anteriormente na Figura 1.

O objetivo foi observar a geração de mensagens ARP Request e ARP Reply durante a comunicação entre os hosts da rede.

O comando utilizado para gerar tráfego foi o seguinte:

```
ping -c 5 8.8.8.8
```

Este comando envia cinco pacotes ICMP Echo Request para o servidor DNS público da Google (8.8.8.8), permitindo observar tanto os pedidos como as respectivas respostas (Echo Reply).

Durante a execução do teste, o sniffer foi colocado em funcionamento na interface de rede ativa, capturando todo o tráfego associado à comunicação ICMP. A análise dos resultados foi realizada de forma comparativa com o Wireshark, ferramenta de referência para inspeção de tráfego de rede.

[15:53:45.794]	wlp62s0	ICMP	1e:ce:51:24:42:31 -> 00:06:91:78:4f:5f	192.168.1.98 -> 8.8.8.8	98 bytes	ICMP echo request
[15:53:45.830]	wlp62s0	ICMP	00:06:91:78:4f:5f -> 1e:ce:51:24:42:31	8.8.8.8 -> 192.168.1.98	98 bytes	ICMP echo reply
[15:53:46.796]	wlp62s0	ICMP	1e:ce:51:24:42:31 -> 00:06:91:78:4f:5f	192.168.1.98 -> 8.8.8.8	98 bytes	ICMP echo request
[15:53:46.831]	wlp62s0	ICMP	00:06:91:78:4f:5f -> 1e:ce:51:24:42:31	8.8.8.8 -> 192.168.1.98	98 bytes	ICMP echo reply
[15:53:47.797]	wlp62s0	ICMP	1e:ce:51:24:42:31 -> 00:06:91:78:4f:5f	192.168.1.98 -> 8.8.8.8	98 bytes	ICMP echo request
[15:53:47.833]	wlp62s0	ICMP	00:06:91:78:4f:5f -> 1e:ce:51:24:42:31	8.8.8.8 -> 192.168.1.98	98 bytes	ICMP echo reply
[15:53:48.798]	wlp62s0	ICMP	1e:ce:51:24:42:31 -> 00:06:91:78:4f:5f	192.168.1.98 -> 8.8.8.8	98 bytes	ICMP echo request
[15:53:48.833]	wlp62s0	ICMP	00:06:91:78:4f:5f -> 1e:ce:51:24:42:31	8.8.8.8 -> 192.168.1.98	98 bytes	ICMP echo reply
[15:53:49.800]	wlp62s0	ICMP	1e:ce:51:24:42:31 -> 00:06:91:78:4f:5f	192.168.1.98 -> 8.8.8.8	98 bytes	ICMP echo request
[15:53:49.816]	wlp62s0	ICMP	00:06:91:78:4f:5f -> 1e:ce:51:24:42:31	8.8.8.8 -> 192.168.1.98	98 bytes	ICMP echo reply

Figura 2: Tráfego ICMP captado pelo nosso sniffer

No.	Time	Source	Destination	Protocol	Length	Info
53	3.709483942	192.168.1.98	8.8.8.8	ICMP	98	Echo (ping) request id=0x0002, seq=1/256, ttl=64 (reply in 54)
54	3.743658912	8.8.8.8	192.168.1.98	ICMP	98	Echo (ping) reply id=0x0002, seq=1/256, ttl=116 (request in 53)
56	4.709743375	192.168.1.98	8.8.8.8	ICMP	98	Echo (ping) request id=0x0002, seq=2/512, ttl=64 (reply in 57)
57	4.729591565	8.8.8.8	192.168.1.98	ICMP	98	Echo (ping) reply id=0x0002, seq=2/512, ttl=116 (request in 56)
69	5.711717915	192.168.1.98	8.8.8.8	ICMP	98	Echo (ping) request id=0x0002, seq=3/768, ttl=64 (reply in 70)
70	5.833110513	8.8.8.8	192.168.1.98	ICMP	98	Echo (ping) reply id=0x0002, seq=3/768, ttl=116 (request in 69)
72	6.711711149	192.168.1.98	8.8.8.8	ICMP	98	Echo (ping) request id=0x0002, seq=4/1024, ttl=64 (reply in 75)
75	6.749723854	8.8.8.8	192.168.1.98	ICMP	98	Echo (ping) reply id=0x0002, seq=4/1024, ttl=116 (request in 72)
76	7.711850542	192.168.1.98	8.8.8.8	ICMP	98	Echo (ping) request id=0x0002, seq=5/1280, ttl=64 (reply in 78)
78	7.736853927	8.8.8.8	192.168.1.98	ICMP	98	Echo (ping) reply id=0x0002, seq=5/1280, ttl=116 (request in 76)

Figura 3: Tráfego ICMP captado pelo Wireshark

A análise comparativa entre o sniffer desenvolvido e o Wireshark permite concluir que a implementação se encontra corretamente funcional na detecção de tráfego ICMP. Observa-se que ambos os sistemas apresentam informação consistente relativamente aos pacotes capturados, nomeadamente na identificação do protocolo ICMP, nos endereços IP de origem e destino, na dimensão dos pacotes e na correspondência entre mensagens Echo Request e Echo Reply.

Desta forma, podemos concluir que os resultados obtidos pelo sniffer estão em conformidade com os apresentados pelo Wireshark, confirmando a correteza da análise efetuada.

5.2 Validação da correta análise do TCP

Para validar o funcionamento do sniffer na detecção e interpretação de segmentos TCP, foi utilizado o ambiente CORE com a topologia de rede previamente apresentada na Figura 1.

O objetivo deste teste foi observar o estabelecimento e encerramento de ligações TCP, permitindo confirmar a correta identificação do *three-way handshake*, bem como das flags associadas à comunicação.

O tráfego foi gerado através da abertura de uma ligação TCP entre dois hosts da rede. Para tal, num terminal do host João foi executado:

```
nc -l -p 5000
```

Num segundo terminal, no host Joana, foi executado:

```
echo "teste" | nc 10.0.1.20 5000
```

Este procedimento força o estabelecimento de uma ligação TCP para a porta 5000, transmitindo dados e encerrando a sessão após a comunicação.

Durante a execução do teste, o sniffer permaneceu ativo na interface de rede em monitorização, capturando todos os segmentos TCP trocados. Em paralelo, o Wireshark foi utilizado como ferramenta de referência para validação dos resultados.

[09:59:02.092]	eth0	TCP	00:00:00:aa:00:01 -> 00:00:00:aa:00:00	10.0.0.20 -> 10.0.1.20	74 bytes	TCP SYN
[09:59:02.092]	eth0	TCP	00:00:00:aa:00:00 -> 00:00:00:aa:00:01	10.0.1.20 -> 10.0.0.20	74 bytes	TCP SYN-ACK
[09:59:02.092]	eth0	TCP	00:00:00:aa:00:01 -> 00:00:00:aa:00:00	10.0.0.20 -> 10.0.1.20	66 bytes	TCP ACK
[09:59:02.092]	eth0	TCP	00:00:00:aa:00:00 -> 00:00:00:aa:00:01	10.0.1.20 -> 10.0.0.20	66 bytes	TCP ACK
[09:59:20.082]	eth0	TCP	00:00:00:aa:00:01 -> 00:00:00:aa:00:00	10.0.0.20 -> 10.0.1.20	66 bytes	TCP FIN

Figura 4: Tráfego TCP captado pelo nosso sniffer

No.	Time	Source	Destination	Protocol	Length	Info
9	8.183777714	10.0.0.20	10.0.1.20	TCP	74	35984 -> 5000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=813038380 TSecr=0 WS=128
10	8.183861895	10.0.1.20	10.0.0.20	TCP	74	5000 -> 35984 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM=1 TSval=452521868 TSecr=813038380 WS=128
11	8.183896592	10.0.0.20	10.0.1.20	TCP	66	35984 -> 5000 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=813038381 TSecr=452521868
12	8.183973460	10.0.0.20	10.0.1.20	TCP	76	35984 -> 5000 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=10 TSval=813038381 TSecr=452521868
13	8.183986537	10.0.1.20	10.0.0.20	TCP	66	5000 -> 35984 [ACK] Seq=1 Ack=11 Win=65152 Len=0 TSval=452521869 TSecr=813038381

Figura 5: Tráfego TCP captado pelo Wireshark

A análise comparativa entre a Figura 4 e a Figura 5 permite confirmar o correto funcionamento do módulo TCP implementado no sniffer.

Observa-se, em ambas as capturas, a sequência típica de estabelecimento de ligação TCP através do *three-way handshake*, composta pelos segmentos SYN, SYN-ACK e ACK. O sniffer desenvolvido conseguiu identificar corretamente esta sequência, bem como os endereços IP de origem e destino envolvidos na comunicação entre os hosts 10.0.0.20 e 10.0.1.20. Após a fase inicial de ligação, o Wireshark apresenta um segmento adicional identificado como PSH, ACK, correspondente à transmissão efetiva de dados enviada pelo comando `echo "teste"`. No sniffer desenvolvido, este segmento surge classificado genericamente como ACK. Esta diferença deve-se ao facto de a implementação atual privilegiar a deteção das flags principais associadas ao estado da ligação, não distinguindo ainda combinações compostas como PSH-ACK. Ainda assim, o segmento foi corretamente capturado e contabilizado como tráfego TCP válido.

Relativamente ao encerramento da sessão, o sniffer identificou corretamente a flag FIN, demonstrando capacidade para reconhecer o término normal da ligação TCP. Este comportamento encontra-se igualmente de acordo com o fluxo esperado para este tipo de comunicação.

Desta forma, conclui-se que o sniffer desenvolvido se encontra corretamente funcional na análise de tráfego TCP, identificando os principais estados da ligação, reconhecendo o estabelecimento e encerramento de sessões e apresentando resultados consistentes com os observados no Wireshark.

5.3 Validação da correta análise do ARP

Para validar o funcionamento do sniffer na deteção e interpretação de pacotes ARP, utilizou-se a topologia montada no ambiente CORE. A rede é composta pelos hosts Joana (10.0.0.1), Joao (10.0.1.20) e Julio, ligados pelo router UMinhoR1, conforme ilustrado na Figura 1.

O tráfego foi gerado a partir de um teste de conectividade, executando o seguinte comando no host Joana:

```
ping -c 5 10.0.1.20
```

Durante a execução, o sniffer capturou o tráfego na interface `eth0`. Em paralelo, o Wireshark foi utilizado como ferramenta de referência para validação dos resultados.

No.	Time	Source	Destination	Protocol	Length	Info
61	98.836453319	00:00:00_aa:00:01	Broadcast	ARP	42	Who has 10.0.0.1? Tell 10.0.0.20
62	98.836473108	00:00:00_aa:00:00	00:00:00_aa:00:01	ARP	42	10.0.0.1 is at 00:00:00:aa:00:00
77	104.251146968	00:00:00_aa:00:00	00:00:00_aa:00:01	ARP	42	Who has 10.0.0.20? Tell 10.0.0.1
78	104.251186685	00:00:00_aa:00:01	00:00:00_aa:00:00	ARP	42	10.0.0.20 is at 00:00:00:aa:00:01

Figura 6: Tráfego ARP captado pelo Wireshark

```

ARP Pair #1
Request: 10.0.0.20 asked → who has 10.0.0.20?
Reply:   10.0.0.1 answered → 10.0.0.20 is at 00:00:00:aa:00:00
RTT:     0.020 ms

-----

ARP Pair #2
Request: 10.0.0.1 asked → who has 10.0.0.1?
Reply:   10.0.0.20 answered → 10.0.0.1 is at 00:00:00:aa:00:01
RTT:     0.040 ms

```

Figura 7: Saída do sniffer para os pares ARP identificados

Na Figura 6 observa-se o tráfego ARP captado pelo Wireshark. Foram gerados dois pares completos de mensagens ARP. O primeiro par (pacotes 61 e 62) corresponde ao pedido para descobrir o endereço físico associado ao destino, seguido da respetiva resposta. O segundo par (pacotes 77 e 78) representa o processo de resolução inverso, permitindo que ambos os intervenientes atualizem as suas tabelas ARP.

Na Figura 7 apresenta-se a saída do sniffer desenvolvido. Os dois pares ARP foram identificados e relacionados com sucesso, sendo possível observar os endereços IP envolvidos, os respetivos endereços MAC e o RTT (*Round-Trip Time*) calculado para cada par (0.020 ms e 0.040 ms, respetivamente).

Os valores de RTT obtidos correspondem ao intervalo entre o *ARP Request* e o *ARP Reply*, refletindo a latência de resolução de endereços na rede local simulada. É possível reparar que, apesar de terem sido enviados vários pacotes ICMP, apenas um número reduzido de mensagens ARP foi observado. Tal deve-se ao facto de as associações IP–MAC serem armazenadas em cache pelos hosts, evitando a redundância de pedidos após a resolução inicial.

Desta forma, conclui-se que o sniffer identifica os pacotes ARP, distingue entre pedidos e respostas, correlaciona os pares e calcula o RTT de forma adequada, apresentando resultados consistentes com os observados no Wireshark.

5.4 Validação da correta análise do IPv4

Para validar o funcionamento do sniffer na deteção e interpretação de pacotes IPv4, utilizou-se a topologia montada no ambiente CORE, com base na configuração apresentada anteriormente na Figura 1.

O objetivo deste teste foi observar a fragmentação de pacotes IPv4, confirmando a correta identificação dos campos do cabeçalho, nomeadamente o campo ID, o *Fragment Offset* e as flags MF e DF.

O tráfego foi gerado a partir do host Joana (10.0.0.20), executando o seguinte comando:

```
ping -c 1 -s 2000 10.0.0.1
```

Este comando envia um pacote ICMP de 2000 bytes para o UMinhoR1 (10.0.0.1), valor superior ao MTU padrão de 1500 bytes, forçando assim a fragmentação ao nível IPv4. Durante a execução, o sniffer capturou o tráfego na interface eth0 do UMinhoR1. Em paralelo, o Wireshark foi utilizado como ferramenta de referência para validação dos resultados.

No.	Time	Source	Destination	Protocol	Length	Info
6	6.253627327	10.0.0.20	10.0.0.1	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=a613) [Reassembled in #7]
7	6.253636985	10.0.0.20	10.0.0.1	ICMP	562	Echo (ping) request id=0x97fd, seq=1/256, ttl=64 (reply in 9)
8	6.253659721	10.0.0.1	10.0.0.20	IPv4	1514	Fragmented IP protocol (proto=ICMP 1, off=0, ID=86b4) [Reassembled in #9]
9	6.253660799	10.0.0.1	10.0.0.20	ICMP	562	Echo (ping) reply id=0x97fd, seq=1/256, ttl=64 (request in 7)

Figura 8: Tráfego IPv4 fragmentado captado pelo Wireshark

```

-----
Packet #003
Route:      10.0.0.20 -> 10.0.0.1
Protocol:   ICMP (Number: 1)
ID:         42515      | Frag Offset: 0
Flags:      [MF=1, DF=0, Res=0]
TTL:        64        | Total Len: 1500   | IHL: 5
Checksum:   0x9af9
Timestamp:  21:16:13.130
-----
Packet #004
Route:      10.0.0.20 -> 10.0.0.1
Protocol:   ICMP (Number: 1)
ID:         42515      | Frag Offset: 185
Flags:      [MF=0, DF=0, Res=0]
TTL:        64        | Total Len: 548    | IHL: 5
Checksum:   0xbdf8
Timestamp:  21:16:13.131

```

Figura 9: Saída do sniffer para os fragmentos IPv4 identificados

Na Figura 8 observa-se o tráfego capturado pelo Wireshark. O pacote ICMP de 2000 bytes foi dividido em dois fragmentos IPv4, ambos partilhando o mesmo identificador (ID=a613). O primeiro fragmento (pacote 6) apresenta MF=1 e offset 0, indicando que existem mais fragmentos a seguir. O Wireshark indica a reassemblagem no pacote 7, correspondente ao Echo Request ICMP completo. O mesmo processo ocorre no sentido inverso, com a resposta do UMinhoR1 para a Joana (pacotes 8 e 9).

Na Figura 9 apresenta-se a saída do sniffer desenvolvido. Os pacotes #003 e #004 correspondem ao datagrama fragmentado com ID=42515, enviado por 10.0.0.20 para 10.0.0.1. O pacote #003 apresenta MF=1 e offset 0, indicando que é o primeiro fragmento e que existem mais a seguir. O pacote #004 apresenta MF=0 e offset 185 (equivalente a 1480 bytes), confirmando que se trata do fragmento final. Ambos partilham o mesmo ID (42515), o mesmo TTL (64) e o mesmo protocolo encapsulado (ICMP), o que permite a sua associação como pertencentes ao mesmo datagrama original.

A correspondência entre os dois datagramas pode ser confirmada através do campo ID: o Wireshark apresenta o valor em hexadecimal (0xa613), enquanto o sniffer apresenta o mesmo valor em decimal (42515), sendo ambos numericamente equivalentes (0xa613 = 42515).

Os resultados obtidos pelo sniffer são consistentes com os observados no Wireshark, confirmando a correta identificação dos campos do cabeçalho IPv4, nomeadamente o ID, o *Fragment Offset*, as flags MF e DF, o TTL e o *checksum*. Conclui-se assim que o sniffer desenvolvido é capaz de capturar e analisar corretamente pacotes IPv4, incluindo situações de fragmentação.

5.5 Validação da correta análise IEEE 802.11 (WiFi)

Para validar o correto funcionamento do módulo de análise de tráfego IEEE 802.11, foi realizada uma captura de pacotes em modo monitor utilizando uma interface wireless configurada para inspeção de frames ao nível da camada 2.

O objetivo deste teste foi verificar a capacidade do sistema em identificar corretamente os diferentes tipos de frames WiFi (Management, Control e Data), bem como a extração de informação relevante associada a redes e dispositivos observados no meio radioelétrico.

Durante a execução do teste, o sistema permaneceu ativo em modo de captura contínua, sendo observados frames IEEE 802.11 provenientes de múltiplos pontos de acesso e dispositivos na área envolvente. A geração de tráfego foi efetuada através da atividade normal de redes WiFi próximas, incluindo transmissão periódica de beacons, processos de associação de clientes e emissão de probe requests.

Os resultados obtidos foram agregados pelo sistema e encontram-se representados na Figura 10.

```
===== WIFI STATUS =====

Traffic Distribution:
management : 235 (81.03%)
control    : 21 (7.24%)
data       : 34 (11.72%)

Connection Quality:
✱ Low (mostly scanning / idle)

Active Networks (BSSIDs):
00:06:91:78:4f:60 -> 74 beacons
00:06:91:78:4f:62 -> 71 beacons
44:df:65:2c:84:01 -> 71 beacons
cc:19:a8:62:d0:c0 -> 8 beacons
cc:19:a8:62:d0:c2 -> 3 beacons
1e:61:b4:55:05:17 -> 1 beacons

Top Devices:
00:06:91:78:4f:60 -> 87 frames
44:df:65:2c:84:01 -> 79 frames
00:06:91:78:4f:62 -> 71 frames
cc:19:a8:62:d0:c0 -> 8 frames
98:77:d5:a0:52:8e -> 7 frames
```

Figura 10: Resumo estatístico do tráfego IEEE 802.11 capturado pelo sniffer

5.5.1 Análise dos resultados

A análise dos resultados permite observar uma distribuição de tráfego dominada por frames de gestão (*management frames*), representando aproximadamente 81% do total

capturado. Este comportamento é esperado em ambientes WiFi reais, uma vez que este tipo de frames inclui beacons, probe requests e mensagens de controlo da infraestrutura da rede.

Os frames de dados representam cerca de 11% do total, refletindo a presença de tráfego de utilizador ativo nas redes detetadas. Já os frames de controlo apresentam uma percentagem reduzida, sendo consistentes com o seu papel auxiliar no acesso ao meio.

Adicionalmente, o sistema foi capaz de identificar múltiplos BSSIDs distintos, correspondentes a diferentes pontos de acesso IEEE 802.11 presentes no ambiente. Para cada BSSID, foi contabilizado o número de beacons recebidos, permitindo caracterizar a presença e atividade das redes sem fios.

No que diz respeito aos dispositivos ativos, o sistema registou múltiplos endereços MAC associados a diferentes níveis de atividade, refletindo a presença de clientes conectados ou em processo de descoberta de redes.

5.5.2 Validação do sistema

Os resultados obtidos demonstram a correta implementação do módulo de análise IEEE 802.11, evidenciando:

- correta classificação dos frames em management, control e data;
- identificação consistente de redes através de BSSID;
- contabilização de beacons como indicador de presença de redes;
- deteção de dispositivos ativos através de endereços MAC;
- coerência global com o comportamento esperado de redes WiFi reais.

Desta forma, conclui-se que o sniffer desenvolvido é capaz de capturar, classificar e agregar corretamente tráfego IEEE 802.11 em tempo real, permitindo uma análise fiável do ambiente wireless envolvente.

5.6 Validação da correta análise do DHCP

Para validar o funcionamento do sniffer na deteção e interpretação de mensagens DHCP, foi forçada uma renovação de endereço IP numa interface de rede real, através dos seguintes comandos:

```
sudo dhcpcd -k eth0
sudo dhcpcd eth0
```

Estes comandos libertam o endereço IP atual e iniciam um novo processo de negociação DHCP com o servidor real da rede, gerando a sequência completa de mensagens DORA (*Discover, Offer, Request, ACK*). Durante a execução, o sniffer e o Wireshark foram colocados em funcionamento simultaneamente na interface `eth0`, permitindo a comparação direta dos resultados.

dhcp						
No.	Time	Source	Destination	Protocol	Length	Info
160	11.546314371	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x145878d9
161	11.572999096	172.17.240.1	172.17.241.173	DHCP	344	DHCP Offer - Transaction ID 0x145878d9
162	11.573460267	0.0.0.0	255.255.255.255	DHCP	342	DHCP Request - Transaction ID 0x145878d9
163	11.593653478	172.17.240.1	172.17.241.173	DHCP	344	DHCP ACK - Transaction ID 0x145878d9

Figura 11: Tráfego DHCP captado pelo Wireshark

```

=== DHCP Flows (per XID) ===
DHCP Transaction XID: 341342425
[ 0.00 ms] DHCP discover | 0.0.0.0 → 255.255.255.255 | 00:15:5d:05:01:01 → ff:ff:ff:ff:ff:ff
[ 26.68 ms] DHCP offer | 172.17.240.1 → 172.17.241.173 | 00:15:5d:51:b8:cc → 00:15:5d:05:01:01
[ 27.15 ms] DHCP request | 0.0.0.0 → 255.255.255.255 | 00:15:5d:05:01:01 → ff:ff:ff:ff:ff:ff
[ 47.34 ms] DHCP ack | 172.17.240.1 → 172.17.241.173 | 00:15:5d:51:b8:cc → 00:15:5d:05:01:01
-----

```

Figura 12: Tráfego DHCP captado pelo sniffer

Na Figura 11 observa-se a sequência DORA completa capturada pelo Wireshark. O processo inicia-se com um *DHCP Discover* enviado em broadcast (0.0.0.0 → 255.255.255.255), ao qual o servidor responde com um *DHCP Offer* contendo o endereço proposto (172.17.241.173). O host confirma com um *DHCP Request* e o servidor finaliza com um *DHCP ACK*. Todas as mensagens partilham o mesmo *Transaction ID* 0x145878d9.

Na Figura 12 apresenta-se o output do sniffer desenvolvido. A transação foi corretamente identificada e agrupada pelo *XID* 341342425. Este valor corresponde ao *Transaction ID* 0x145878d9 apresentado pelo Wireshark, sendo numericamente equivalentes — o Wireshark apresenta o valor em hexadecimal enquanto o sniffer o apresenta em decimal (0x145878d9 = 341342425). Esta correspondência confirma que ambas as ferramentas identificaram a mesma transação DHCP. O sniffer apresenta ainda os tempos relativos de cada mensagem em relação ao *Discover*, permitindo observar que o processo de negociação completo demorou aproximadamente 47 ms.

Os resultados obtidos pelo sniffer são consistentes com os observados no Wireshark, confirmando a correta identificação e agrupamento das mensagens DHCP, bem como o cálculo dos tempos relativos de cada fase da negociação. Conclui-se assim que o sniffer desenvolvido é capaz de capturar e analisar corretamente o protocolo DHCP, identificando a sequência DORA e associando corretamente as mensagens pertencentes à mesma transação.

6 Análise em Interface de Rede Real

Para complementar os testes realizados no ambiente controlado (CORE), o sniffer foi também executado numa interface de rede real, permitindo observar o comportamento do tráfego em condições reais.

A captura foi realizada numa interface ativa (ex.: wlan0), em modo contínuo, durante a utilização normal da rede (navegação web, acesso a serviços online, etc.).

6.1 Observações

Durante a captura, foi possível identificar diversos tipos de tráfego, nomeadamente:

- Tráfego **TCP**, associado a ligações HTTP/HTTPS, com múltiplos handshakes e trocas de dados;
- Pacotes **ICMP**, utilizados em testes de conectividade e controlo de rede;
- Mensagens **ARP**, resultantes da resolução de endereços na rede local;
- Tráfego **DHCP**, observado em momentos de renovação de endereço IP;
- Frames **IEEE 802.11**, evidenciando atividade de redes WiFi próximas.

Verificou-se que o volume de tráfego em ambiente real é significativamente superior ao ambiente simulado, com múltiplas comunicações concorrentes e maior diversidade de protocolos.

6.2 Resultados

O sniffer demonstrou capacidade para:

- Capturar pacotes em tempo real sem perdas significativas;
- Identificar corretamente os protocolos presentes;
- Aplicar os handlers implementados em cenários reais;
- Produzir estatísticas consistentes com o comportamento observado.

Estes resultados confirmam a aplicabilidade da ferramenta em ambientes reais, para além do contexto académico e simulado.

7 Obstáculos Encontrados

Durante o desenvolvimento do sniffer, foram encontrados diversos desafios técnicos, tanto ao nível da captura como da análise dos pacotes:

- **Identificação do protocolo principal (`top_protocol`)** A determinação do protocolo principal de cada pacote revelou-se não trivial, uma vez que um único pacote pode conter múltiplas camadas (por exemplo, IPv4 + ICMP ou IPv4 + TCP). Foi necessário definir uma lógica consistente que permitisse identificar corretamente o protocolo dominante, garantindo simultaneamente que os handlers apropriados eram invocados (por exemplo, processar um pacote tanto no handler IPv4 como no handler ICMP).
- **Captura em ambiente real com elevado volume de tráfego** A transição do ambiente simulado (CORE) para uma interface real introduziu um volume significativamente superior de pacotes, com múltiplas comunicações simultâneas. Este cenário exigiu maior eficiência no processamento, levando à introdução de um modo com menor verbosidade, que reduz o overhead associado à formatação e output dos pacotes, melhorando assim o desempenho em cenários de elevado volume de tráfego.

- **Captura de pacotes em modo monitor** A configuração de interfaces WiFi em modo monitor revelou-se dependente do sistema operativo e drivers, dificultando a captura de frames IEEE 802.11.
- **Reconstrução e correlação de fluxos** A associação entre pacotes relacionados (ex.: pedidos e respostas em ICMP e ARP, ou fases do handshake TCP) exigiu a definição de chaves únicas e tratamento de situações em que pacotes podem chegar fora de ordem ou não serem capturados.
- **Gestão de fragmentação IPv4** A identificação e agrupamento correto de fragmentos implicou compreender detalhadamente os campos do cabeçalho IP (ID, offset, flags).
- **Interpretação de protocolos complexos** Protocolos como DHCP e TCP exigem análise de múltiplos estados, tornando a implementação mais complexa do que protocolos simples como ARP.

8 Limitações da Aplicação Desenvolvida

Apesar dos resultados obtidos, o sniffer apresenta algumas limitações:

- **Análise parcial de flags TCP** A implementação não distingue todas as combinações de flags (ex.: PSH-ACK), tratando apenas os estados principais.
- **Dependência da biblioteca Scapy** A análise dos pacotes depende das funcionalidades disponibilizadas pela biblioteca Scapy, podendo existir limitações na dissecação de certos protocolos ou campos específicos.
- **Limitações na captura de tráfego WiFi** A captura de frames IEEE 802.11 depende do suporte da interface de rede para modo monitor, o que nem sempre está disponível ou corretamente configurado.
- **Escalabilidade e uso de memória** A utilização de estruturas em memória para armazenar pacotes e dados de análise pode tornar-se ineficiente em capturas de longa duração ou com elevado volume de tráfego.
- **Ausência de interface gráfica** O sistema funciona exclusivamente em linha de comandos, o que limita a visualização e exploração interativa dos dados quando comparado com ferramentas como o Wireshark.
- **Análise limitada a protocolos implementados** A aplicação apenas suporta análise detalhada dos protocolos para os quais existem handlers implementados, não sendo extensível automaticamente a novos protocolos.

9 Conclusão do Trabalho Efetuado

O presente trabalho permitiu o desenvolvimento de um *packet sniffer* funcional, capaz de capturar, analisar e interpretar tráfego de rede em múltiplos protocolos da pilha TCP/IP, bem como de redes sem fios IEEE 802.11.

A arquitetura modular adotada revelou-se fundamental para a organização do sistema, permitindo separar claramente as responsabilidades entre captura, parsing, análise por protocolo, estatísticas e apresentação de resultados. Esta abordagem facilitou a extensibilidade do projeto e a integração de novos módulos de forma consistente.

Ao longo do desenvolvimento, foram implementados handlers específicos para protocolos como ICMP, TCP, ARP, DHCP, IPv4 e WiFi, permitindo não apenas a identificação dos pacotes, mas também a reconstrução de interações relevantes, como o *three-way handshake* TCP, a sequência DORA do DHCP ou o emparelhamento request/reply em ICMP e ARP. Os testes realizados em ambiente controlado (CORE) e em interface de rede real permitiram validar o correto funcionamento do sistema, demonstrando a sua capacidade de operar em diferentes cenários e sob diferentes volumes de tráfego. A comparação com ferramentas de referência, como o Wireshark, confirmou a consistência dos resultados obtidos.

Apesar dos resultados positivos, foram identificadas algumas limitações, nomeadamente ao nível da profundidade da análise de certos protocolos, da escalabilidade em ambientes de elevado tráfego e da ausência de uma interface gráfica.

Em conclusão, o trabalho cumpriu os objetivos propostos, fornecendo uma ferramenta funcional para análise de tráfego de rede e contribuindo para uma melhor compreensão do funcionamento dos principais protocolos das camadas de rede, transporte e aplicação.